Agent Harness：模型和环境之间的胶水

语言模型

给一串文字
→ 接着往下写

harness

← 环境的状态 → 写成文字

→ 模型的输出 → 解析成动作

→ 执行动作

← 结果写回去

环境

终端、棋盘
真实发生的事



作业说明开头

三部分：同一个循环跑三个领域

SWE-bench 题

第二部分 · 上下文压缩

修 bug 的 agent → fix.patch → 修好的棋盘服务 ← 下棋的 agent

第三部分 · 工具：落子

同一个 Agent 基类 · 同一个循环



作业说明流程图

评分：只重放，不重跑模型

56	34	8
私有测试	重放 patch 和运行记录	报告

评分不重新调用模型 · 只重放你提交的记录

作业提供 · 不许改

while 循环

不管正常结束
还是中途出错：
✓ 写运行记录
✓ 关掉沙箱



作业说明 Grading

我的解题顺序

1 先读测试

说明写给人看，测试规定得更精确

`tests/`

2 记下起点

一开始：11 失败 · 4 通过

`11 失败` `4 通过`

3 先离线、后花钱

离线测试不调模型、不开沙箱

`make test` → `Modal + 模型`

4 每部分一个提交

出了问题能退回去

`938eeed` `c91bf81` `dffbc6b`



开发日志

● 循环
工具
踩坑
修 bug
压缩
下棋
实验

每一步发过去的是一串消息

system	长期规则	排第一条，只有一条
user	任务	接着一条
assistant	模型的输出	模型每一轮的输出
tool	工具调用的结果	紧跟发起调用的那条
assistant	模型的输出	动作
tool	工具调用的结果	结果
assistant	模型的输出	动作
tool	工具调用的结果	结果

动作

↓ 越排越长

模型



作业说明 Part 1

循环
工具
踩坑
修 bug
压缩
下棋
实验

build_prompt：每一步重新拼这串消息

base.py

```python
def build_prompt(self) -> list[dict[str, Any]]:
    messages: list[dict[str, Any]] = [
        {"role": "system", "content": self.system_prompt},
        {"role": "user", "content": self.task_prompt},
    ]
    if self.working_memory:
        messages.append(...)
    messages.extend(deepcopy(self.history))
    return messages
```

✓ 天然碰不到原始指令

每次请求时重新拼出来

- `system` 长期规则
- `user` 任务

history：只存任务之后发生的事

- `assistant` 模型的输出
- `tool` 工具调用的结果

⑥ 压缩只改 history



base.py

循环
工具
踩坑
修 bug
压缩
下棋
实验

run：一个 while 循环，三处判断

base.py

```
while not self.finished:
    if self.steps_taken >= self.step_limit:
        raise StepLimitError(...)
    self.maybe_compact_context()
    action = self._assistant_message(self.query_language_model())
    self.history.append(action)
    tool_calls = action.get("tool_calls", [])
    if not tool_calls:
        self.history.append({"role": "user", "content": NO_TOOL_CALL_NUDGE})
        continue
    observations = self.execute_tool_calls(tool_calls)
    self.history.extend(self._linked_observations(tool_calls, observations))
```

① 先检查步数上限，再做压缩

反过来：到了上限那一步，还会多花一次模型调用

② 只回了一段文字，没调任何工具

什么都没执行

保留这条回复 · 再补一句 · 接着循环

③ 评分表里专门有一项测这个

user 你这一轮没调用工具，什么都没发生



循环
工具
踩坑
修 bug
压缩
下棋
实验

这个情况不是假设

后面下棋时：沙箱断了

示意

assistant 服务器连不上

user 你这一轮没调用工具，什么都没发生

assistant 服务器连不上

user 你这一轮没调用工具，什么都没发生

assistant 服务器连不上

… 一次次只回一段话

要是当成错误直接退出

这一局当场就中断了 ✗

保留这条回复，然后接着循环

run 里的第二处判断 ✓



开发日志

循环
工具
踩坑
修 bug
压缩
下棋
实验

第三处：工具结果怎么和调用对上

base.py

```
self.history.extend(self._linked_observations(tool_calls, observations))
```

③



按 id 全局找：配错

配到别的轮次上去

按位置配对

这一轮的第几个调用 → 这一轮的第几条结果

✓ 一个调用一条结果



循环
工具
踩坑
修 bug
压缩
下棋
实验

修代码的 agent：两个工具

`>_` **execute** 在沙箱里跑命令

✓ **send_message** 任务完成

提交之后不再执行

示意

assistant | 1 send_message {"message": "…"} | 2 execute {"command": "…" | 3 run_tests {}

1 tool 已发送，会话结束

```
Message sent. The session is complete.
```

2 tool 没执行：任务已经用 send_message 提交

```
Not executed: the task was already submitted with send_message.
```

3 tool 没执行：任务已经用 send_message 提交

```
Not executed: the task was already submitted with send_message.
```



作业说明 TODO 1.3

循环
• 工具
踩坑
修 bug
压缩
下棋
实验

技能：渐进式披露

只给目录 · 要用时再给全文

system

长期规则 ……

<skills>
submit-task：任务完成时按步骤提交，提交前必须先调用这个技能
</skills>

目录

assistant `invoke_skill {"name": "submit-task"}`

tool 完整内容：① 生成 patch ② 检查 ③ 提交

技能再多，目录也只是多几行，不会一上来塞满上下文

SKILL.md tasks/code-skills/submit-task/

```
---
name: submit-task
description: 任务完成时按步骤提交，
提交前必须先调用这个技能
---
```

第 1 步 生成 patch 文件

第 2 步 检查 patch

第 3 步 提交

名字重复 → 报错

开头描述缺了、格式不对 → 报错

一个技能都没有 → 提示词里不出现 patch 文件



作业说明 TODO 1.4

循环
工具
踩坑
修 bug
压缩
下棋
实验

提交技能：分三步走

第 1 步

生成 patch 文件

只放改过的源代码

不放测试和临时脚本

→

第 2 步

单独读一遍

确认内容对

→

第 3 步

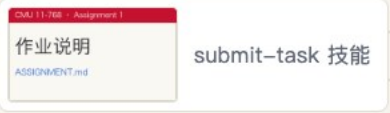
提交

`send_message`

检查之后又改了 patch → **重新检查**

一旦提交 → **不能再做任何事**

✓ 后面真跑时，agent 都照做了



submit-task 技能

循环
工具
踩坑
修 bug
压缩
下棋
实验

公开测试过了，不等于对了

公开测试 `make test`

15 通过 ✓

私有测试会测的情况：没覆盖

自检脚本 · 41 项 **41 / 41 ✓**

本地起一个真实的棋盘服务

- 坏参数
- 网络断开
- 并行落子
- 坏技能文件
- 重复技能
- 刷新棋盘

不放进 tests/：作业不许改

落子时网络断了

✓ 返回一条错误

× ~~抛异常~~

技能目录里两个同名技能

✓ 加载时就报错

× ~~悄悄用掉其中一个~~

作业点名：私有测试会测，公开测试没测

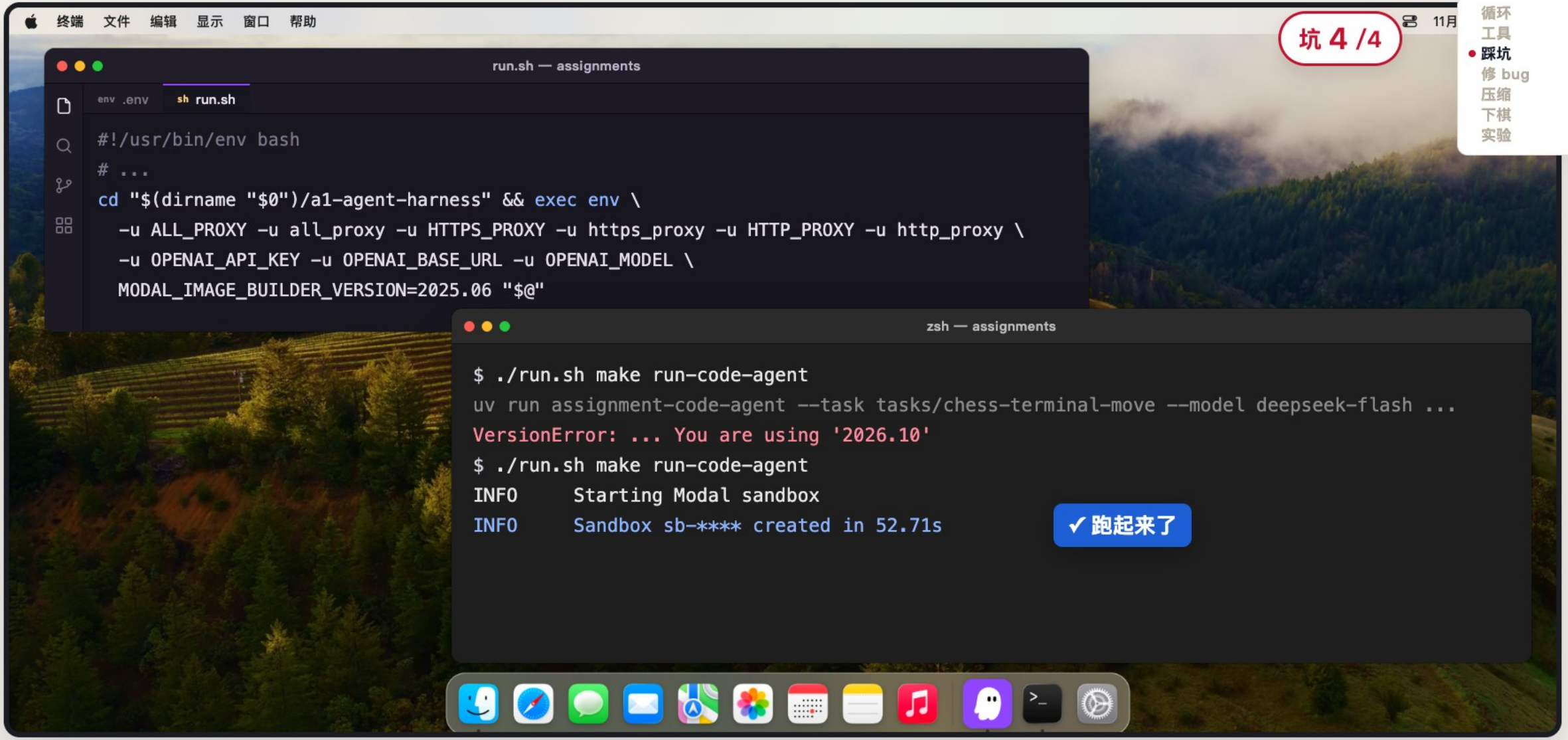


开发日志 · 验证



循环
工具
踩坑
修 bug
压缩
下棋
实验

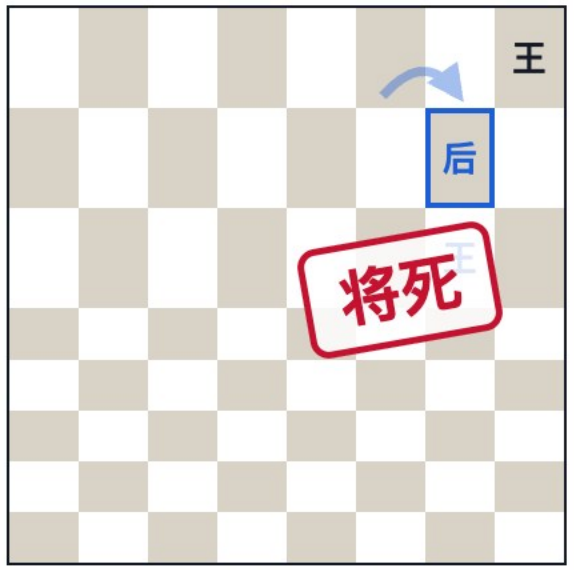
开发日志 · 环境


终端 文件 编辑 显示 窗口 帮助
11月
坑 4 /4
循环
工具
踩坑
修 bug
压缩
下棋
实验
run.sh — assignments
env .env
sh run.sh
#!/usr/bin/env bash
...
cd "$(dirname "$0")/a1-agent-harness" && exec env \
-u ALL_PROXY -u all_proxy -u HTTPS_PROXY -u https_proxy -u HTTP_PROXY -u http_proxy \
-u OPENAI_API_KEY -u OPENAI_BASE_URL -u OPENAI_MODEL \
MODAL_IMAGE_BUILDER_VERSION=2025.06 "$@"
zsh — assignments
$ ./run.sh make run-code-agent
uv run assignment-code-agent --task tasks/chess-terminal-move --model deepseek-flash ...
VersionError: ... You are using '2026.10'
$ ./run.sh make run-code-agent
INFO Starting Modal sandbox
INFO Sandbox sb-**** created in 52.71s
✓跑起来了

开发日志 · 真跑

循环
工具
踩坑
修 bug
压缩
下棋
实验

题目：白方一步将死，服务返回 500



7k/6Q1/6K1/8/8/8/8/8 b - - 1 1

请求 POST /api/move

```
{"move": "f7g7"}
```

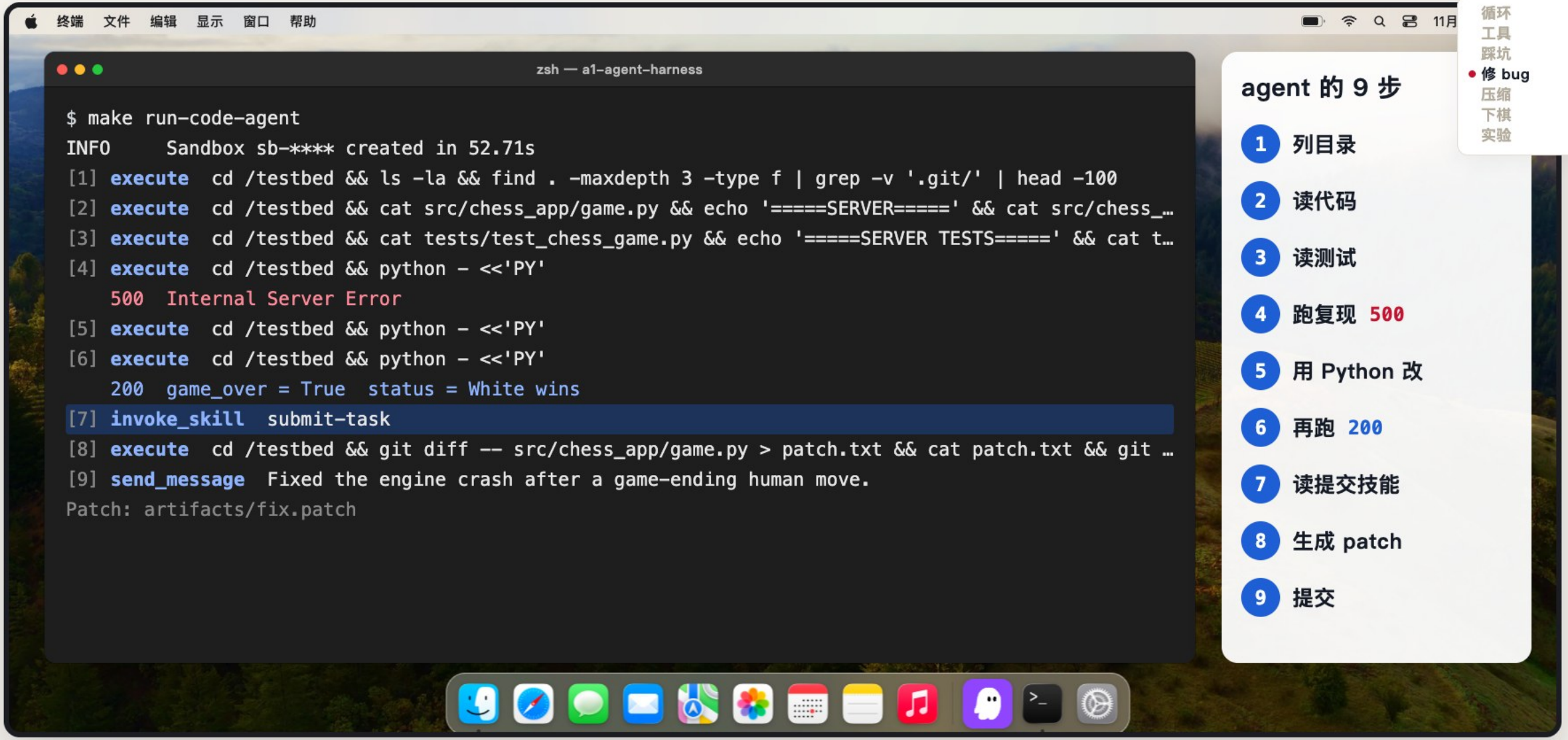
500 Internal Server Error

棋盘服务内部





作业题目 Part 1


终端 文件 编辑 显示 窗口 帮助
11月
zsh — a1-agent-harness
$ make run-code-agent
INFO Sandbox sb-**** created in 52.71s
[1] execute cd /testbed && ls -la && find . -maxdepth 3 -type f | grep -v '.git/' | head -100
[2] execute cd /testbed && cat src/chess_app/game.py && echo '=====SERVER=====' && cat src/chess_…
[3] execute cd /testbed && cat tests/test_chess_game.py && echo '=====SERVER TESTS=====' && cat t…
[4] execute cd /testbed && python - <<'PY'
500 Internal Server Error
[5] execute cd /testbed && python - <<'PY'
[6] execute cd /testbed && python - <<'PY'
200 game_over = True status = White wins
[7] invoke_skill submit-task
[8] execute cd /testbed && git diff -- src/chess_app/game.py > patch.txt && cat patch.txt && git …
[9] send_message Fixed the engine crash after a game-ending human move.
Patch: artifacts/fix.patch
agent 的 9 步
1 列目录
2 读代码
3 读测试
4 跑复现 500
5 用 Python 改
6 再跑 200
7 读提交技能
8 生成 patch
9 提交
循环
工具
踩坑
修 bug
压缩
下棋
实验

循环
工具
踩坑
修 bug
压缩
下棋
实验

patch 只改了一处

fix.patch

```
@@ -75,9 +75,10 @@ class ChessGame:
             engine_move: str | None = None
-            reply = self._choose_engine_move()
-            engine_move = reply.uci()
-            self._record_and_push(reply)
+            if self._board.outcome(claim_draw=True) is None:
+                reply = self._choose_engine_move()
+                engine_move = reply.uci()
+                self._record_and_push(reply)
```

全新环境重放 patch

原本失败的测试
FAIL_TO_PASS 1/1 ✓

原有测试
PASS_TO_PASS 9/9 ✓

RESOLVED ✓

判断：对局有没有结束 `outcome(claim_draw=True)`

示意

是不是将死

- 将死
- 逼和
- 重复局面导致的和棋

→ 同样不该让引擎再走

我们的代码 fix.patch

循环
工具
踩坑
修 bug
压缩
下棋
实验

提示词每一步都在变长

system 长期规则

user 任务：django 的一个 bug

工作记忆 模型自己写的总结，替换旧的原文

assistant 第 4 步的动作

tool 结果





作业说明 Part 2

循环
工具
踩坑
修 bug
压缩
下棋
实验

先看这道题：django 的 bulk_update

django__django-15368 示意

```
obj.name = F('other')
Model.objects.bulk_update([obj], ['name'])
```

↓ × 不认这个表达式

数据库

id	name
1	'F(other)'

数据库里写进去的是字符串 'F(other)'

FAIL_TO_PASS 1 个 · PASS_TO_PASS 29 个

1
原本失败 → 要通过

29
原有测试 → 不能改坏

django 代码量大 → agent 要读的东西多 → 提示词涨得快



SWE-bench 题目

循环
工具
踩坑
修 bug
压缩
下棋
实验

压缩的三条规矩

① 原样保留

`system` `user`

只总结旧的部分

`assistant` `tool` `assistant` `tool` `tool`

② 最近一步连同结果留着

`assistant` `tool`

③ api_prompts · api_responses：评分要用，不许动

每次请求前
`maybe_compact_context()`

→ 估提示词长度 → 到阈值了? → 压缩

上一次请求时模型服务报回来的真实长度 + 之后新增部分的粗略估计

```
return actual_previous + added_since_previous_request
```



循环
工具
踩坑
修 bug
压缩
下棋
实验

按"步"切：切点只能落在两步之间

system user 工作记忆 assistant tool

一起交给模型，合并成新的一份

上一份工作记忆

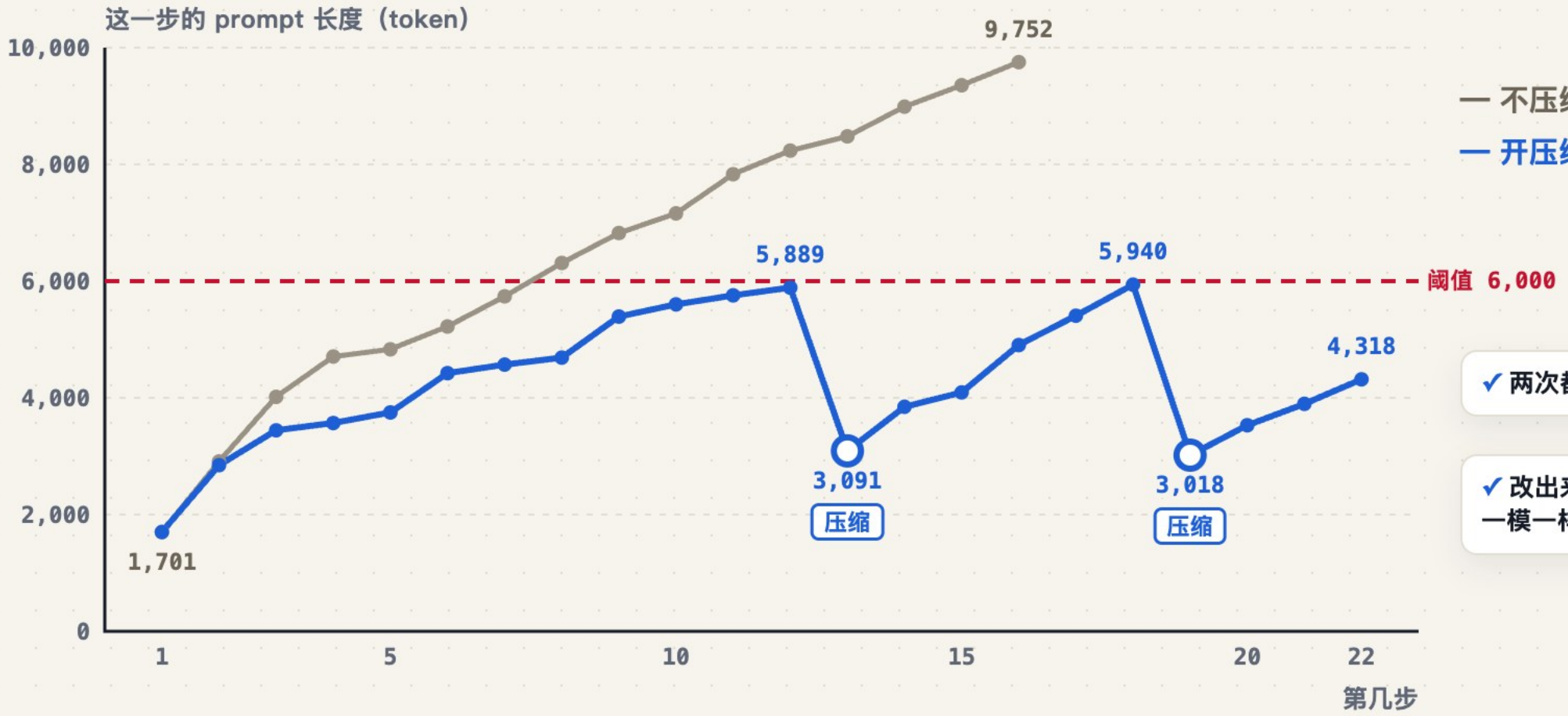
总结是空的 → 这次不删

宁可这次不压缩，也不能把上下文丢了

```
if summary and split > 0:
```



提示词长度：不压缩一路涨，开压缩是锯齿



✓ 两次都把题做对了

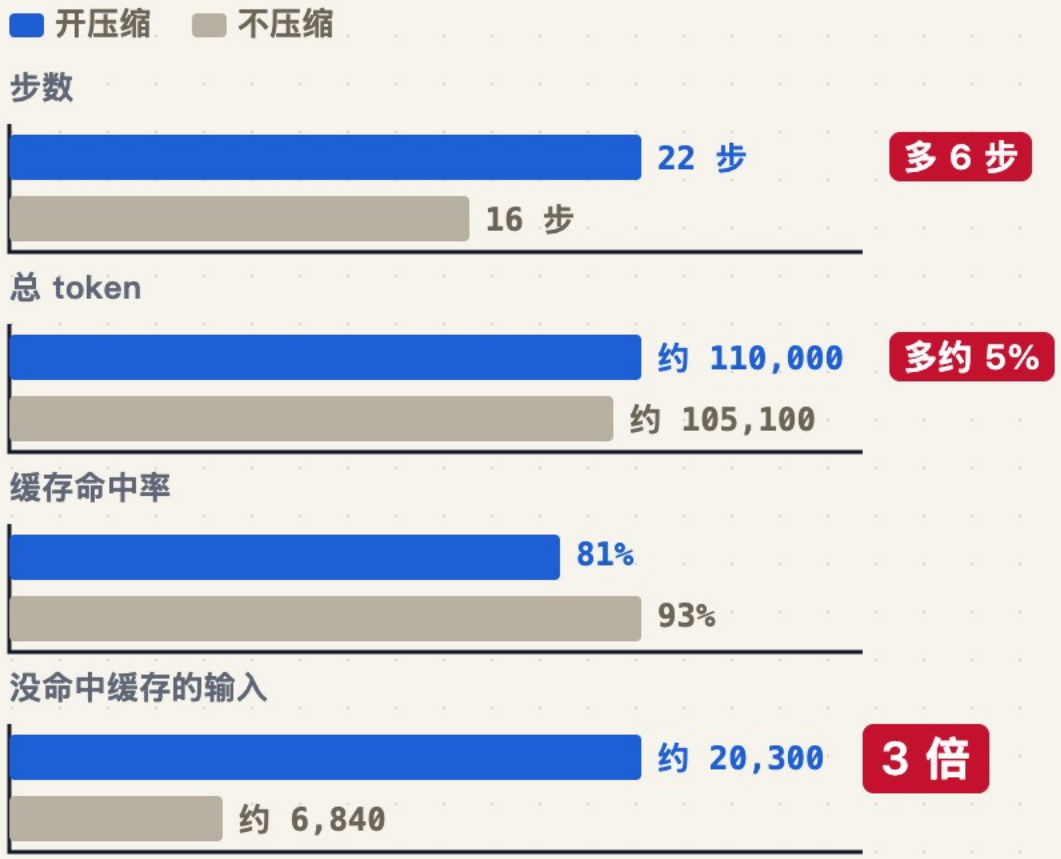
✓ 改出来的代码
一模一样



token 用量报告

循环
工具
踩坑
修 bug
压缩
下棋
实验

算总账：开压缩反而更贵



每一步的请求

第 n 步　开头一大段　新

第 n+1 步　开头：和上一步一模一样　新　新

相同的开头缓存起来 → 算得快、便宜得多

压缩之后　总结 · 没命中　最近一步　新

开头改写成了总结 → 缓存作废

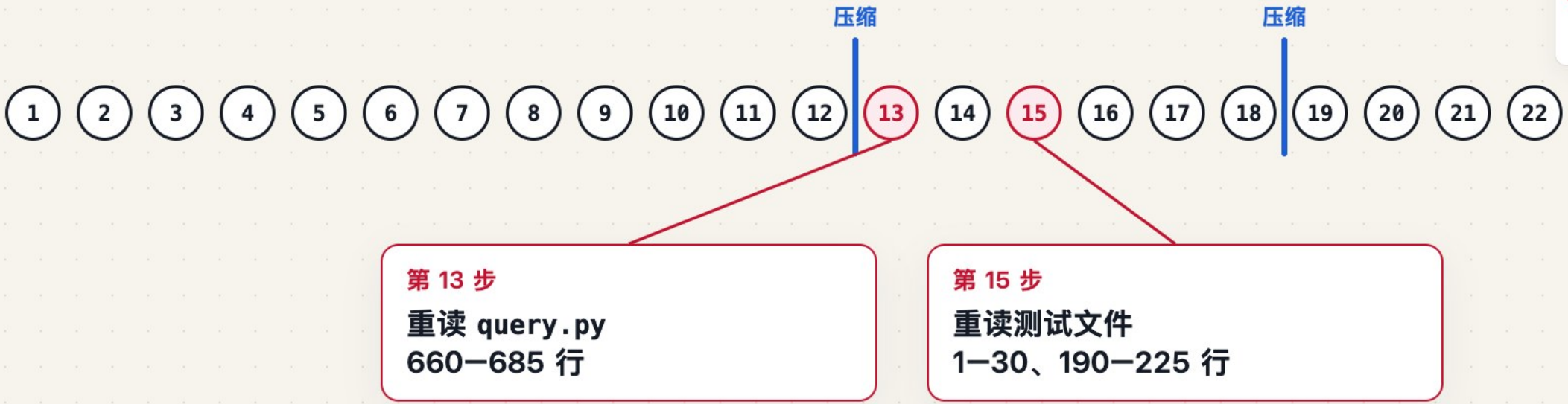
写总结的那次调用：几乎全是新内容



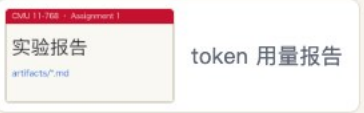
token 用量报告

循环
工具
踩坑
修 bug
压缩
下棋
实验

压缩之后，agent 回头重读



✓ 总结记下了做过什么　✕ 没记下它当时依赖的原文



循环
工具
踩坑
修 bug
压缩
下棋
实验

压缩换来的是一个上下文的上限

任务长到会撑爆窗口时

救命的

在一道十几步就能做完的题上

额外的调用

作废的缓存

丢掉的细节

这部分怎么评分

✓ 至少触发一次压缩

✓ 上下文明显变短

✓ patch 通过测试

三条都满足 · 步数多一点不扣分



token 用量报告

循环
工具
踩坑
修 bug
压缩
• 下棋
实验

每走完一步，模型看到的观察

tool 结果 · 白 e2e4、黑回 e7e5 之后

```
<chess_state>
status: Your turn
turn: white
in_check: False
game_over: False
fen: rnbqkbnr/pppp1ppp/8/4p3/4P3/8/PPPP1PPP/RNBQKBNR w KQkq - 0 2
human_move: e2e4
engine_move: e7e5
board:
recent_history: 1:e4 2:e5
legal_moves: a2a3 a2a4 b1a3 b1c3 b2b3 b2b4 c2c3 c2c4 d1e2 d1f3 d1g4 d1h5 d2d3 d2d4 e1e2 f1a6 f1b5 f1c4 f1d3 f1e2
             f2f3 f2f4 g1e2 g1f3 g1h3 g2g3 g2g4 h2h3 h2h4
</chess_state>
```

对局状态
轮到谁走
有没有被将军
对局结束没有
局面的标准记法
白方和黑方刚走的那两步
字符画的棋盘 →
最近八步棋

```
    a b c d e f g h
  +-----------------+
8 | r n b q k b n r | 8
7 | p p p p . p p p | 7
6 | . . . . . . . . | 6
5 | . . . . p . . . | 5
4 | . . . . P . . . | 4
3 | . . . . . . . . | 3
2 | P P P P . P P P | 2
1 | R N B Q K B N R | 1
  +-----------------+
    a b c d e f g h
```

默认还带全部合法着法 · ⑧ 的实验就是把这份列表拿掉做对比



作业说明 Part 3

改过棋盘之后，同一条回复里的落子都退回

chess_agent.py

```
board_changed = False
...
    elif name in ("play_move", "run_python") and board_changed:
        content = _error(
            f"{name} was not executed: an earlier call in this response "
        ...
    elif name == "play_move":
        content = self._play(arguments)
        board_changed = not content.startswith("<chess_error>")
    elif name == "simulate_move":
        content = _simulate_move(self.chess_client, arguments)
    elif name == "run_python":
        content, board_changed = self._run_snippet(arguments)
```

同一条回复里，棋盘改过之后：

play_move 会改棋盘

→ 退回，按新局面重选

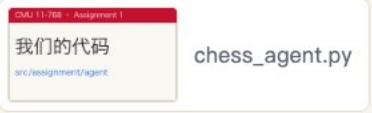
run_python 会改棋盘

→ 退回，按新局面重选

simulate_move 只推演

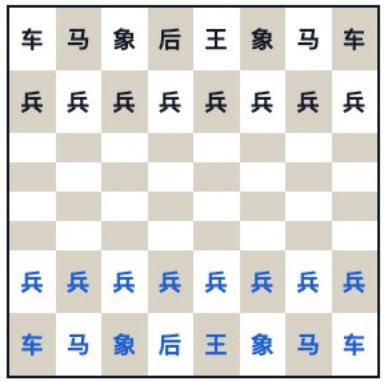
→ 照常执行

循环
工具
踩坑
修 bug
压缩
• 下棋
实验



chess_agent.py

simulate_move：只推演，不落子



rnbqkbnr/pppppppp/8/8/8/8/PPPPPPPP/RNBQKBNR w KQkq - 0 1

真实棋盘

示意

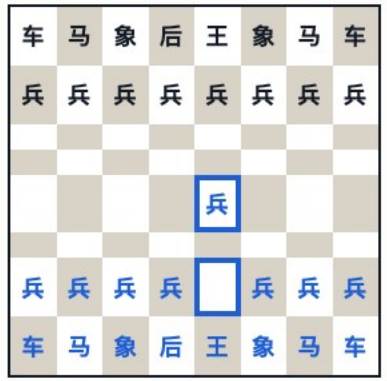
`simulate_move(局面)`

`legal_moves: a2a3 a2a4 b1a3 …`

`simulate_move(局面, e2e4) →`

`fen: 走完之后的局面`

不碰真实棋盘 · 不触发机器人



rnbqkbnr/pppppppp/8/8/4P3/8/PPPP1PPP/RNBQKBNR b KQkq - 0 1

推演棋盘

作业原文 · TODO 3.1.b（_play_move）

Catch any errors raised by the tool ... Cover malformed JSON arguments, arguments that are not an object, a missing or non-string fen, a non-string move, a position or move the server rejects, and a transport failure.

play_move 没有这个参数 ← TODO 3.3（_simulate_move）里一字不差的同一句



作业说明 TODO 3.3

循环
工具
踩坑
修 bug
压缩
• 下棋
实验

run_python：让模型写程序调工具

tool 输出 + 最新局面



chess_tools.py

```
except Exception as exc:
    return _chess_error(f"The Python sandbox failed: {exc}")
if result.get("returncode") != 0:
    ...
    return _chess_error(f"The Python sandbox failed: {detail}")
# The runner prints exactly one JSON object; pass it through unchanged.
stdout = result.get("stdout")
```

代码自己报错了

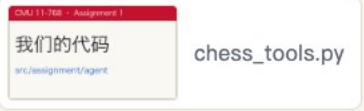
比如变量名写错

算正常运行 · 错误原样还给模型

沙箱本身出了问题

命令根本没跑起来

工具失败 · <chess_error>



chess_tools.py

循环
工具
踩坑
修 bug
压缩
• 下棋
实验

Kiro 文件 编辑 显示 窗口 帮助

run_python · 第 4 步 — part3-python-skill-trajectory.json

py run_python · 第 4 步

```
VAL = {'p':100,'n':320,'b':330,'r':500,'q':900,'k':0}
CENT = {'d4','e4','d5','e5'}
...
        return 100000 if w=='white' else (-100000 if w=='black' else 0)
...
    moves = sorted(cur['legal_moves'])
    for m in moves:
        aw = simulate_move(fenn, m)
        ...
            s = min(score_pos(simulate_move(aw['fen'], r)) for r in replies) if replies else score_pos(aw)
        if best_score is None or s > best_score:
            best_score, best = s, m
...
play_move(b)
```

兵 100 · 马 320 · 象 330 · 车 500 · 后 900

中间四格 ±12

将死 ±100,000



part3 运行记录



开发日志

part3 运行记录

循环
工具
踩坑
修 bug
压缩
下棋
• 实验

两个模型的速度：差得很远



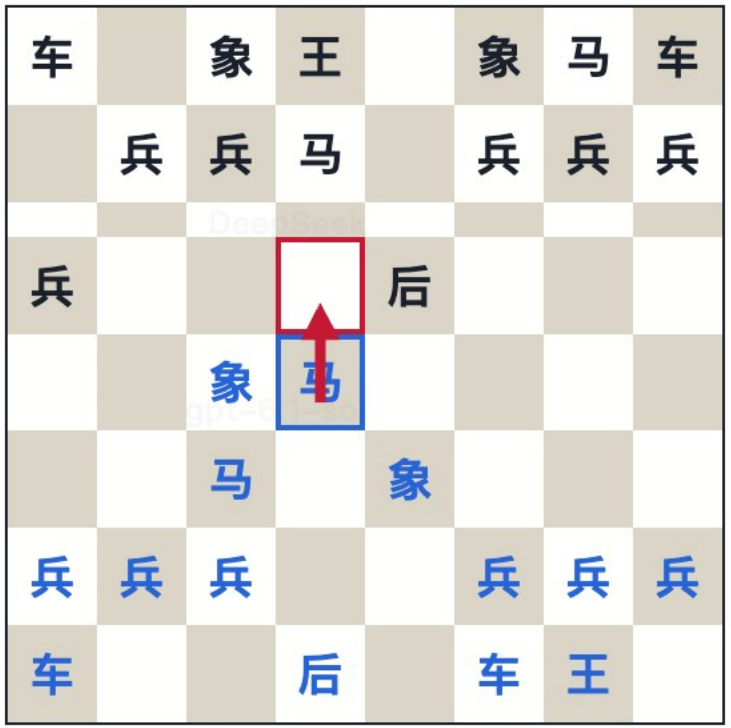
下得快的那个：非法着法一次都没有



观测实验报告

循环
工具
踩坑
修 bug
压缩
下棋
• 实验

唯一那次非法：把马当成兵走了



r1bk1bnr/1ppn1ppp/8/p3q3/2BN4/2N1B3/PPP2PPP/R2Q1RK1 w - - 0 10

给它的合法着法列表 第 11 步 · DeepSeek · 棋盘 + 合法着法

a1b1	a1c1	a2a3	a2a4	b2b3	b2b4	c3a4
c3b1	c3b5	c3d5	c3e2	c3e4	c4a6	c4b3
c4b5	c4d3	c4d5	c4e2	c4e6	c4f7	d1b1
d1c1	d1d2	d1d3	d1e1	d1e2	d1f3	d1g4
d1h5	d4b3	d4b5	d4c6	d4e2	d4e6	d4f3
d4f5	e3c1	e3d2	e3f4	e3g5	e3h6	f1e1
f2f3	f2f4	g1h1	g2g3	g2g4	h2h3	h2h4

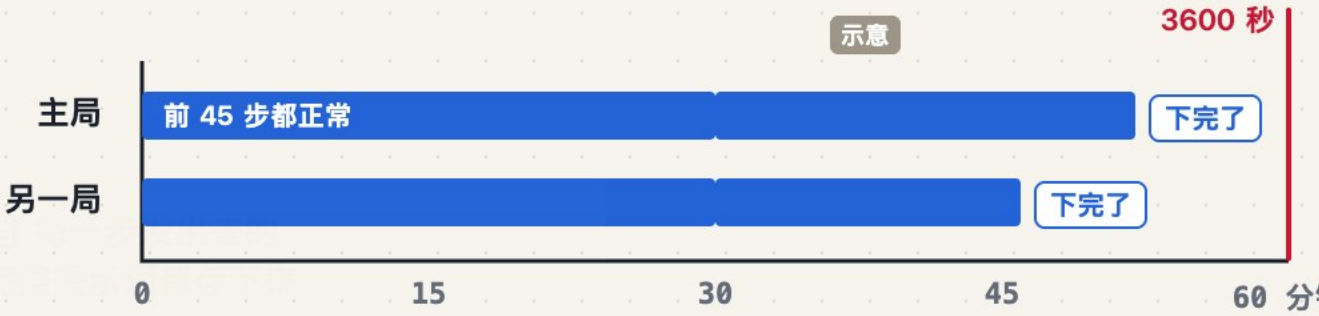
~~d4d5~~ 根本不在列表里

d4 上站着的是白方的马：想把它往前推一格



观测实验报告

翻车一：沙箱到期，空转 155 步



第 46 步起

盘服务 | 连不上棋盘服务 | 连不上棋盘服务 | 连不上棋盘服务 | 连不上棋盘服务 | 只回文字 | 示意

空转 **155** 步

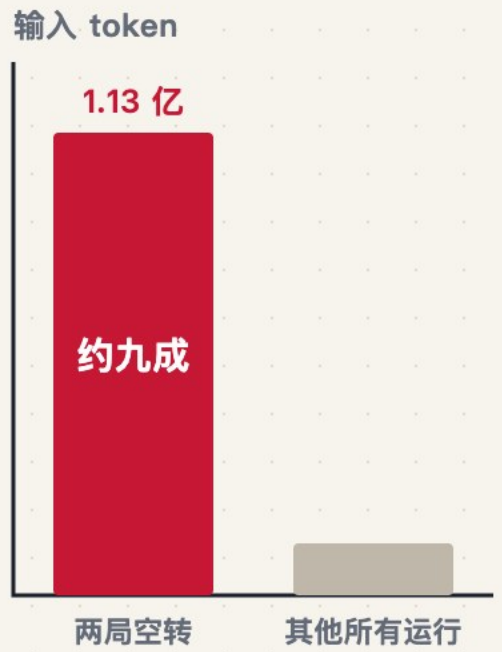
StepLimitError · 200 步

125 次落子
全是“连不上棋盘服务”

29 次只回了一段话
服务器一直断开，没法继续

✓ 程序没有崩
网络错误交还给模型处理

✕ 没有熔断
“连续失败几次就停”



好在大部分命中了缓存



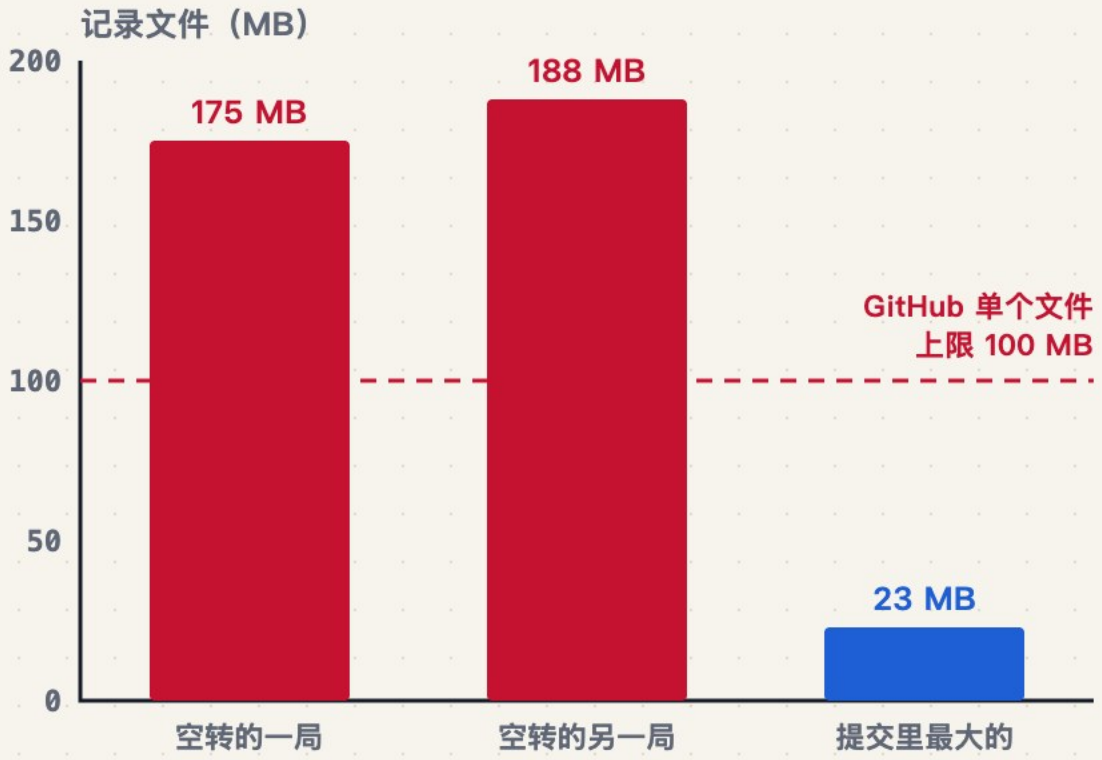
开发日志

循环
工具
踩坑
修 bug
压缩
下棋
• 实验

运行记录：超过 GitHub 单个文件上限

每一步发出去的
完整提示词都存下来

那两局空转的棋，记录文件越存越大

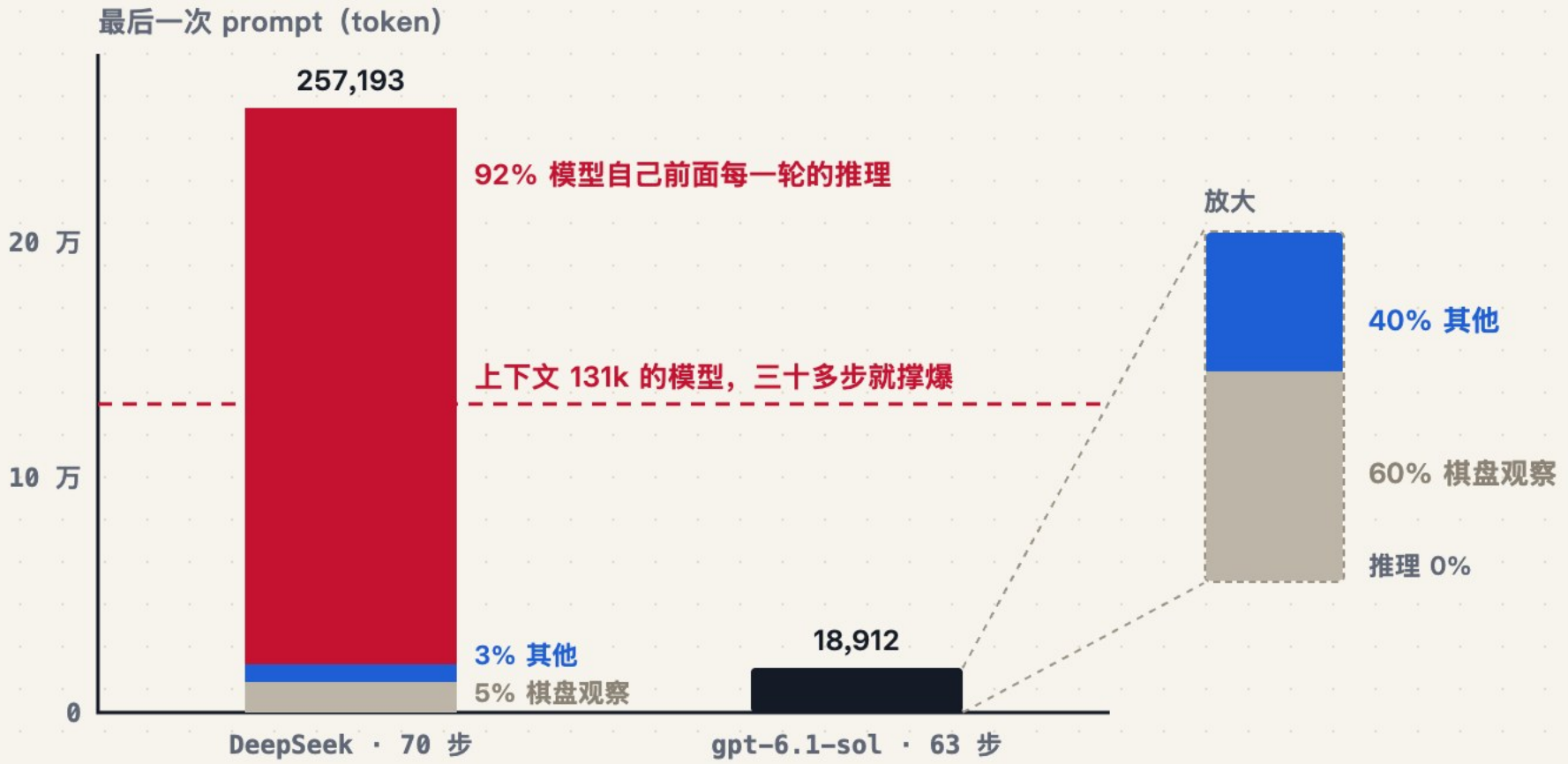




开发日志

循环
工具
踩坑
修 bug
压缩
下棋
• 实验

翻车二：上下文里九成是模型自己的推理





开发日志

循环
工具
踩坑
修 bug
压缩
下棋
• 实验

如果重来一次：四件事一开始就做

示意

① 给循环加熔断

连续几次都连不上环境，就停下来报告

□ □ □ □ □ 连续 5 次连不上 → 停下报告

示意

② 推理内容只留最近几轮

不全部发回去

全部发回去

只留最近几轮

示意

③ 沙箱寿命按每一步的耗时去估

而不是用默认值

35 秒 × 200 步 → 约 2 小时 30 分钟的沙箱不够

④ 统计结果前，先分清错误的类型

第一版统计脚本把网络错误也算成了非法着法

非法着法　网络错误



精读稿 · 开发日志

第一

一个循环，多个领域

修代码和下棋用同一个 run

第二

错误还给模型

再配一个熔断

第三

消息结构不能断

按位置配对 · 按步切

第四

上下文是设计出来的

推理发不发回去，差出十几倍

第五

公开测试过了不算完

56 分在私有测试里



开发日志 · 实验报告